CONCLAVE: Agentic AI Without a Language Model — Ensemble Machine Learning as the Cognitive Core for Connectivity-Denied Autonomous Systems

Aranya Mahmud

***Abstract*—Agentic artificial intelligence (AI) systems observe their environment, take decisions and act through software tools, and most of them are built around a large language model (LLM) running on a cloud server or on powerful local hardware. In disaster response this is a serious problem: during floods, cyclones and earthquakes the communication network is often among the first infrastructures to fail, and the devices left in the field are low-cost and battery-powered, so an LLM-based agent may become unavailable exactly when it is most needed. To address this problem, this paper proposes CONCLAVE, an agentic AI framework in which a heterogeneous ensemble of classical machine learning models replaces the LLM as the decision-making core. The framework combines hashed text and sensor features, a soft-voting committee, a decomposition of aleatoric and epistemic uncertainty, a calibrated gate that decides whether to act, gather evidence, abstain or escalate, and a NumPy-only runtime for low-cost boards. On two public datasets, a flood-response simulator and emulated hardware profiles, CONCLAVE selects the correct tool with 90.8% accuracy in 4.0 ms, reaches 74.3% weighted F1 on crisis tweets, and keeps a positive return of 33.5 when 73% of the steps are offline, whereas a cloud-dependent agent falls to -127.3. Against eight other machine-learning-based agents it obtains the highest return and the fewest casualties, and its runtime needs 32 MB of memory and an estimated 1.7 ms per decision on a Raspberry Pi Zero 2 W. Hence, reliable agentic decision-making for disaster response is possible without the internet and on inexpensive hardware.**

***Index Terms*—Agentic AI, ensemble learning, disaster response, edge computing, offline autonomy, uncertainty quantification, tool use, Raspberry Pi.**

I. Introduction

FLOODS, cyclones and earthquakes regularly disrupt human life, damage infrastructure and cause heavy economic loss, and densely populated, low-lying regions are among the most exposed. Effective disaster management depends on timely decisions in every phase of the disaster lifecycle, from early warning to rescue and recovery. Over the last decade, artificial intelligence (AI) has made important contributions to this field. Machine learning (ML) models have been used for environmental monitoring [1] and for locating and deploying relief goods [2]; multi-agent and case-based reasoning systems have been proposed for flood early warning [3]; ensemble learning has been combined with Internet-of-Things (IoT) sensors for flood detection [4]; and, more recently, language and vision models have been applied to post-disaster damage assessment [5], to the extraction of flood information from text and images [6], [7], and to emergency decision support [8], [9]. Recent reviews document such applications across the full disaster lifecycle [10], [11].

At the same time, a new class of AI systems, known as *agentic* AI, has emerged. An AI agent does not only classify or predict: it observes its environment, decides what to do, and acts through software tools in a loop. Since Toolformer [12] showed that a language model can learn when to call external tools, agent design has converged on a common pattern in which a large language model (LLM) performs the reasoning and the other components — memory, tools and planners — are built around it. The pattern has been extended with deliberate search over reasoning paths [13] and with collaboration among several LLM agents [14]. Disaster management has adopted it quickly: agentic systems now automate post-disaster damage assessment and reporting [5], coordinate response activities [15], and support emergency decision-making [9], [8].

However, these systems share a dependency that is critical in disaster settings. Many of them send their reasoning to hosted models, for example GPT-4 in DisasTeller [5] and ChatGPT-4o-mini through a cloud application programming interface (API) in a community flood-sensor system [16]. During a flood or a cyclone, cellular and backhaul networks are often among the first infrastructures to fail, and the hardware that remains in the field is usually low-cost and battery-powered. Under such conditions a cloud-backed agent cannot reason at all. A local LLM avoids the network, but it needs a large amount of memory and computing power, which inexpensive devices such as the Raspberry Pi do not provide.

The research community has responded mainly by *compressing* the language model. A broad survey of small language models (SLMs) describes the compression techniques that make them viable on edge hardware, together with their use in specialised, domain-specific settings [17], and in disaster response this line has produced genuinely offline systems: ResQConnect, for example, provides offline mobile guidance from an edge-deployed SLM in under 500 ms per response [15]. Nevertheless, compressed models still occupy hundreds of megabytes to gigabytes, need hundreds of milliseconds per response, and inherit the failure modes of genera-

Department of Computer Science and Engineering, Daffodil International University, Dhaka, Bangladesh. E-mail: aranyamahmud29@gmail.com

Code, data-preparation scripts, and all result artefacts are available at https://github.com/Aranya69/conclave.

tive models, such as unreliable output that must be constrained by external knowledge [8] and urgency judgements that change across languages for identical scenarios [18].

Many decisions of an autonomous disaster responder, however, are not open-ended. Once the set of tools is registered and the possible actions are enumerated, the remaining problem is classification, ranking and sequential control under uncertainty, which is exactly the problem that classical ML solves efficiently. There is already evidence for this view: on crisis-tweet classification, compact semi-supervised models can outperform very large LLMs used in zero-shot mode [19]. Therefore, the research question of this paper is as follows: *can an agent whose decision-making core is an ensemble of classical ML models, without any language model, provide reliable agentic behaviour for disaster response when there is no internet and only low-cost hardware is available?*

To answer this question, we propose CONCLAVE, an agentic AI framework whose cognitive core is a heterogeneous committee of classical ML models. The committee selects actions and tools; the disagreement among its members gives an estimate of uncertainty; and a calibrated gate uses this estimate to decide whether the agent should act, gather more evidence, abstain or escalate the decision to a human. The complete agent runs offline and fits on a board such as the Raspberry Pi Zero 2 W.

The main contributions of this paper are summarised as follows:

1) **An LLM-free agentic AI framework.** CONCLAVE realises the complete agent loop — perception, action selection, open-vocabulary tool routing, symbolic planning, case-based episodic memory and safety constraints — with a committee of classical ML models. The trained disaster agent occupies 232 KB and takes about 2.5 ms per decision on a CPU.
2) **Uncertainty-aware control.** The committee's predictive entropy is decomposed into aleatoric and epistemic parts, which are mapped to four control actions (act, gather, abstain and escalate) with thresholds calibrated by selective prediction rather than set by hand.
3) **A complete data engineering and evaluation protocol.** The framework is evaluated on two public datasets, BFCL[1] and HumAID[2], on a flood-response simulator with controllable network outages, and on emulated low-cost hardware. A NumPy-only edge runtime reduces the memory needed on the device from 185 MB to 32 MB without changing any decision.
4) **A comprehensive comparison with other agents.** Under no-internet and low-cost-hardware conditions, CONCLAVE is compared with eight other machine-learning-based agents, a cloud-dependent agent and fixed threshold rules. It gives the highest closed-loop return and the lowest casualties, while remaining one of the cheapest agents to run.

[1]Berkeley Function Calling Leaderboard, maintained by S. G. Patil *et al.* at UC Berkeley: https://gorilla.cs.berkeley.edu/leaderboard.html.

[2]HumAID was released by F. Alam, U. Qazi, M. Imran and F. Ofli (ICWSM 2021) and is available at https://huggingface.co/datasets/QCRI/HumAID-all.

The rest of this paper is organised as follows. Section II reviews the related literature. Section III describes the proposed methodology, including data collection, data engineering, the models, the uncertainty mechanism and the evaluation protocol. Section IV presents the experimental results and the comparison with other agents. Section V discusses the findings and their practical implications, Section VI states the limitations and future work, and Section VII concludes the paper.

II. Literature Review

Intelligent decision support for disasters did not begin with language models. Earlier work used multi-agent and case-based reasoning systems for flood early warning [3], machine learning for environmental monitoring and relief logistics [1], [2], and ensemble learning with IoT sensors [4]. More recent work uses LLM-based agents that reason over text, images and tools [10], [11]. These newer systems are the most capable, but they also depend the most on connectivity and computing power. This section reviews the literature in five parts and then identifies the research gap.

A. LLM-centred agent architectures

The dominant agent architectures place an LLM at the centre of a reason–act loop. Tree of Thoughts [13] extends chain-of-thought prompting to a deliberate search over reasoning paths and raises the success rate of GPT-4 on the Game of 24 from 4% to 74%; Toolformer [12] learns in a self-supervised way which APIs to call; and Chain of Agents [14] distributes long-context reasoning across worker and manager LLMs. The same core has been proposed as a replacement for reinforcement-learning policies, selecting actions directly from textual descriptions of the state [20]. On the other hand, a systematic evaluation has shown that LLMs generate executable plans poorly on their own [21], which suggests caution in placing such a model at the centre of a safety-critical control loop.

B. LLMs and agents in disaster management

Surveys. Reviews now cover LLM applications in disaster management [10], generative AI more broadly [11], AI methods for locating and deploying relief goods [2], and AI in environmental monitoring, where a bibliometric analysis of 4,762 publications identifies disaster management as a principal application [1].

Multi-agent response systems. DisasTeller coordinates four vision–language agents around GPT-4 for damage assessment, alerting and recovery planning [5]; the Scenario–Learning–Decision framework couples LLM agents with prospect-theoretic modelling for wildfire response [9]; ResQConnect combines agentic retrieval with adaptive resource routing [15]; and LLM-driven agents have been used to simulate population behaviour during disasters [22].

Perception and information extraction. LLMs have been applied to the multimodal extraction of flood water depth [7], to geospatial intelligence extraction from Chinese disaster

text [23], to fine-grained flood information [6], to urban flood detection from 340 CCTV feeds, where four multimodal LLMs were compared on both accuracy and operational cost [24], and to the reconstruction of landslide events from witness reports and site imagery [25]. Knowledge graphs are a recurring device for constraining LLM output: they ground flood-risk dialogue and GIS interaction [26], build disaster storylines from global news [27], and steer emergency decision support, motivated explicitly by the unreliability of unconstrained LLM output [8]. Portable flood sensors have also been paired with an LLM to personalise community warnings [16].

C. Crisis text classification

HumAID is the standard benchmark for humanitarian tweet triage, and its authors report transformer and classical baselines on its official split, with which we compare in Section IV-B. LLM-guided semi-supervised learning improves low-resource crisis classification, and compact semi-supervised models can outperform large zero-shot LLMs [19]. For multilingual crisis messages, LLM urgency judgements vary widely across languages, whereas human judgements do not [18].

D. Compact and offline models

The SLM line of work [17] shares our diagnosis — many workloads are narrow and specialised — but it keeps the language model. Parameter-efficient fine-tuning with QLoRA is widely used to obtain lightweight LLMs, including outside the disaster domain, for example in smart-contract vulnerability detection [28]. In disaster settings, this line has reached genuine offline operation: the edge SLM of ResQConnect answers in under $500\,\text{ms}$ on commodity smartphones [15].

E. Classical ensembles, case-based reasoning and uncertainty

The closest architecture to ours is the flood-warning framework of Sen et al. [4], which aggregates a deep network, a random forest and a nearest-neighbour classifier over IoT sensor data and pairs them with an LLM for social-media text. Two design choices separate it from CONCLAVE. First, its committee is aggregated by *hard majority vote*, which discards the spread of the members' probabilities that our gate uses as an epistemic signal. Second, it keeps an LLM for text and relies on fog and cloud layers. Case-based reasoning has been combined with multi-agent systems for flood early warning at the conceptual level [3]; CONCLAVE's episodic memory is a case-based-reasoning component that is implemented and evaluated. It is well established that ensembles improve accuracy and give useful uncertainty estimates [29], [30], [31]; the split of predictive uncertainty into aleatoric and epistemic parts that we use follows the standard entropy-based formulation [32]; and selective prediction, in which a model may decline to predict, has its own literature [33], [34].

F. Research gap

Table I compares CONCLAVE with the systems closest to it. Three gaps can be identified from the above review. First, offline capability, where it exists, is obtained by shrinking a language model, which still needs hundreds of megabytes and hundreds of milliseconds per decision. Second, escalation to a human, where it exists, is a workflow decision rather than a calibrated function of model uncertainty. Third, the ensemble-based disaster systems discard the members' probability spread and still depend on fog or cloud layers. To the best of our knowledge, no existing work removes the language model from the agent loop altogether, uses committee disagreement as a control signal, and evaluates the resulting agent under network outages and on low-cost hardware. CONCLAVE is designed to fill this gap.

III. Methodology

This section describes the proposed methodology step by step. Fig. 1 shows the overall workflow, which consists of six stages: (1) data collection, (2) data engineering, (3) feature engineering, (4) model training, (5) the offline agent loop, and (6) evaluation. The problem is first formulated in Section III-A; the data and their preparation are described in Sections III-B–III-C; the features, models and uncertainty mechanism are explained in Sections III-D–III-K; the complete decision loop is given in Algorithm 1 and Fig. 2; and the simulator, the edge runtime, the compared agents and the evaluation metrics are described in Sections III-M–III-Q.

A. Problem formulation

We consider an agent that interacts with an environment in discrete time steps $t = 1, \ldots, T$. At each step the agent receives an observation $o_t = (s_t, r_t)$, where $s_t \in \mathbb{R}^d$ is a vector of d numeric sensor readings and r_t is an optional free-text report. The agent chooses an action a_t either from a finite set $\mathcal{A} = \{a_1, \ldots, a_K\}$ of K registered tools or from three control actions: *gather* more evidence, *abstain*, or *escalate* to a human operator. The environment returns a reward R_t, and the aim is to learn a policy $\pi(a \mid o)$ that maximises the expected episode return $\mathbb{E}\big[\sum_{t=1}^{T} R_t\big]$ under three operational requirements: (i) the policy must run without any network access; (ii) it must fit within the memory and computing budget of a low-cost board; and (iii) it must recognise when its own decision should not be trusted. In CONCLAVE, π is realised by a committee of M classical classifiers, and the confidence of the policy is estimated from their agreement.

B. Data collection

The framework is evaluated on four test beds, summarised in Table II. Each test bed isolates one capability that an offline agent needs, so that each capability can be measured on its own. Two of them are public datasets. BFCL v3 pairs each user request with the specifications of several candidate tools (functions); the agent must pick the tool that serves the request, or recognise that none does. HumAID contains tweets posted during real disasters, each labelled with one of ten humanitarian categories, such as *requests or urgent needs* and *infrastructure and utility damage*; the agent must sort each message into its category. The other two test beds were

TABLE I
SYSTEMS CLOSEST TO CONCLAVE. "NOT REPORTED" MEANS THE PAPER DOES NOT DESCRIBE THE PROPERTY, NOT THAT THE SYSTEM LACKS IT.

System	Reasoning core	Task	Network at inference	Escalation	Footprint
DisasTeller [5]	GPT-4 + 4 LVLM agents	Damage assessment	Hosted LLM	Not reported	Cloud
Ou et al. [16]	ChatGPT-4o-mini	Community flood alerts	Cloud API	Not reported	Cloud
Sen et al. [4]	DL+RF+KNN vote + LLM	Flood detection	Fog + cloud	Not reported	Fog/cloud
ResQConnect [15]	Agentic RAG + edge SLM	Response coordination	Offline guidance tier	Human oversight	Smartphone
Rashid et al. [3]	MAS + CBR	Flood warning	Not reported	Not reported	Conceptual
CONCLAVE (ours)	**ML committee, no LM**	**Agentic control + triage**	**Offline**	**Calibrated gate**	**232 KB**

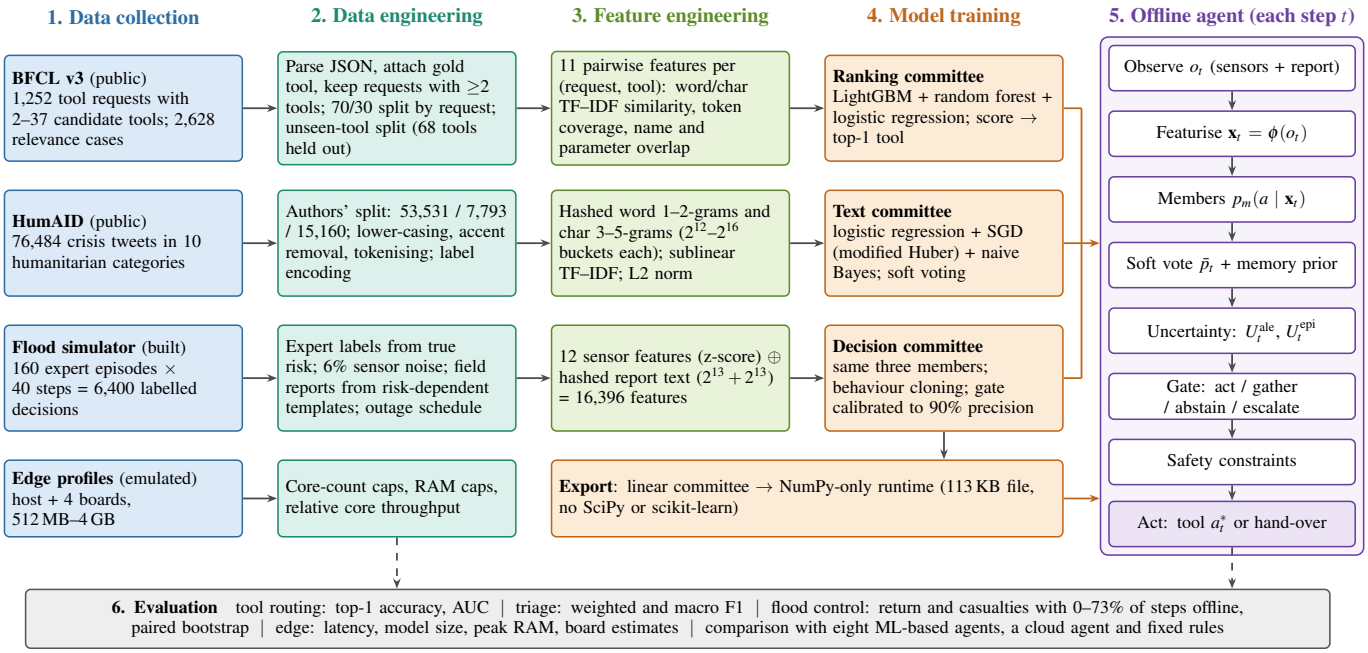


Fig. 1. Overall methodology of the proposed CONCLAVE framework. Each public dataset or test bed passes through data engineering and feature engineering before training a committee of classical ML models; the trained committees are deployed in an offline agent loop, and every stage is evaluated on accuracy, closed-loop performance and resource cost.

built for this study, because no public dataset measures what they measure: a flood-response simulator tests decisions over time while the network fails, and emulated hardware profiles test whether the agent fits, and runs fast enough, on low-cost boards (Table III).

C. Data preprocessing and engineering

1) BFCL. Each BFCL record contains a user request and a list of candidate function specifications in JSON format, and the ground-truth call is stored in a separate file. We join each request with the name of its ground-truth function, keep only the user turns of the conversation, and remove requests whose gold function is not among the candidates or which offer fewer than two candidates. This gives 200 requests from the `multiple` split and 1,052 from the `live_multiple` split (1,252 in total), with 2 to 37 candidate tools per request and 3.8 on average. Each request is expanded into (request, tool) pairs, labelled 1 for the gold tool and 0 otherwise. The data are split by request (70% for training and 30% for testing) and never by pair, so that no request appears on both sides. To test generalisation to new tools, we also build an *unseen-tool* split: one third of the tools of `live_multiple` (68 tools) are held out, test requests target only held-out tools, and training requests never contain them. For abstention, 1,118 irrelevant requests from the two irrelevance splits are combined with 1,510 relevant requests, giving 2,628 cases (42.5% irrelevant), which are split 70/30 with stratification.

2) HumAID. HumAID provides the authors' official training, validation and test sets of 53,531, 7,793 and 15,160 tweets. We keep this split, so that our results can be compared directly with the baselines reported by the dataset authors. The text is lower-cased and accents are removed; word tokens are extracted with a regular expression for words of at least two characters, and character n-grams are taken within word boundaries. No tweet is removed, and the ten class labels are encoded as integers. The class distribution is highly imbalanced: the largest class (*rescue, volunteering or donation effort*) has 14,891 training tweets, whereas the smallest (*missing or found people*) has only 250. For this reason, both weighted and macro F1 are reported.

3) Simulator data. The training data of the flood agent are generated by an expert controller (Section III-M) that can see the hidden flood risk. We run 160 episodes of 40 steps with

TABLE II
THE FOUR TEST BEDS. TWO ARE PUBLIC DATASETS AND TWO WERE BUILT FOR THIS STUDY; EACH ISOLATES ONE CAPABILITY.

Test bed	What is used	Capability tested
BFCL v3 (public dataset)	1,252 requests, each offering two or more candidate tools (3.8 on average); 2,628 requests for abstention, 43% of which no tool fits	Choosing the right tool, or declining when none fits (Sec. IV-A)
HumAID (public dataset)	76,484 tweets in 10 humanitarian categories, with the authors' train/validation/test split	Sorting crisis messages by category (Sec. IV-B)
Flood simulator (built here)	40-step episodes in 6 outage settings, with the network down for 0–73% of steps; 120 test episodes per setting	Acting over time while the network is down (Sec. IV-C)
Edge profiles (built here)	The host machine and 4 emulated boards with 512 MB to 4 GB of RAM, from a Raspberry Pi Zero 2 W to a Jetson Nano	Fitting and running fast on small hardware (Sec. IV-D)

TABLE III
EMULATED HARDWARE PROFILES. RELATIVE THROUGHPUT ρ IS THE APPROXIMATE SINGLE-CORE SPEED OF EACH BOARD RELATIVE TO THE HOST, TAKEN FROM VENDOR AND PUBLISHED FIGURES; IT IS USED ONLY TO ESTIMATE ON-DEVICE LATENCY.

Profile	Cores	RAM	Clock (MHz)	ρ	TDP (W)
Host (x86-64)	2	3.8 GB	—	1.00	15.0
Jetson Nano	4	4 GB	1,430	0.26	10.0
Raspberry Pi 4B	4	4 GB	1,500	0.28	6.4
Raspberry Pi 3B+	4	1 GB	1,400	0.17	5.0
Raspberry Pi Zero 2 W	4	512 MB	1,000	0.11	2.5

different random seeds and record, at every step, the agent's noisy observation together with the expert's action. This gives 6,400 labelled decisions for behaviour cloning. The sensor readings are corrupted by multiplicative Gaussian noise with a standard deviation of 6%, and every observation contains a short field report whose wording depends on the true risk. A further 60 episodes (2,400 decisions) with unseen seeds form the test set for one-step accuracy, and 120 episodes with unseen seeds are used for closed-loop evaluation.

4) Edge profiles. For each profile in Table III, the number of CPU threads is limited to the board's core count and, where the operating system allows, the process address space is limited to the board's RAM. On-device latency is then estimated as described in Section III-P.

5) Data quality check. Before selecting the test beds, we audited a widely used Bangladesh flood-prediction dataset. A standard ensemble reaches 99.6% accuracy on it, but a depth-1 decision stump on rainfall alone scores 98.4%, and 92% of the rows are positive. Its "Flood?" label is therefore essentially a rainfall threshold, and any model can recover it. Since such a benchmark cannot separate a good model from a trivial one, it was not used.

D. Feature extraction

Hashed text features. A report r is converted into a multiset $\mathcal{G}(r)$ of word n-grams ($n = 1, 2$) or character n-grams ($n = 3, 4, 5$). Instead of storing a vocabulary, every n-gram g is mapped directly to a column by the hashing trick:

$$j(g) = \big|h(g)\big| \bmod D, \qquad c_j(r) = \sum_{g \in \mathcal{G}(r)} \mathbb{I}\big[j(g) = j\big], \quad (1)$$

where h is the 32-bit MurmurHash3 function, $D = 2^b$ is the number of hash buckets, $c_j(r)$ is the number of n-grams of r that fall into bucket j, and $\mathbb{I}[\cdot]$ is the indicator function. Separate spaces of D_w buckets (words) and D_c buckets (characters) are used. The counts are then re-weighted by sublinear term frequency and inverse document frequency (TF–IDF):

$$\tilde{u}_j(r) = \big(1 + \ln c_j(r)\big)\,\mathrm{idf}_j, \qquad \mathrm{idf}_j = \ln \frac{1+N}{1+\mathrm{df}_j} + 1, \quad (2)$$

for $c_j(r) > 0$ and $\tilde{u}_j(r) = 0$ otherwise, where N is the number of training documents and df_j is the number of training documents in which bucket j is non-zero. Each vector is normalised to unit length, $u(r) = \tilde{u}(r)/\|\tilde{u}(r)\|_2$. Because there is no stored vocabulary, the featuriser keeps only the two idf vectors and its size is fixed by D_w and D_c. A conventional fitted vocabulary on HumAID would occupy about 98 MB, whereas the hashed featuriser is constant-size and is tuned by a single parameter b, which we sweep in Section IV-B.

Sensor features. Each sensor reading is standardised with the mean μ_i and standard deviation σ_i of the training data,

$$z_i(s) = \frac{s_i - \mu_i}{\sigma_i}, \qquad i = 1, \ldots, d, \quad (3)$$

with $\sigma_i = 1$ for a constant feature. The final feature vector concatenates the two views:

$$\mathbf{x}_t = \phi(o_t) = \big[\, z(s_t)\,;\; u^w(r_t)\,;\; u^c(r_t) \,\big] \in \mathbb{R}^{\,d + D_w + D_c}. \quad (4)$$

For the flood agent, $d = 12$ and $D_w = D_c = 2^{13}$, which gives 16,396 features; for HumAID, only the text view is used, with $D_w = D_c \in \{2^{12}, 2^{14}, 2^{16}\}$.

Pairwise ranking features. For tool routing, a request q and a tool τ are described jointly by an 11-dimensional vector $\varphi(q, \tau)$. The tool text concatenates the tool's name, its description, and the names and descriptions of its parameters. The features are: (1) word TF–IDF similarity between the request and the tool text; (2) character TF–IDF similarity; (3) the fraction of request tokens found in the tool text; (4) the fraction of tool tokens found in the request; (5) the fraction of tool-name tokens found in the request; (6) the number of required parameters; (7) the number of parameters; (8) the length of the tool text; (9) the fraction of parameter names mentioned in the request; (10) the fraction of required parameter names mentioned; and (11) the number of request tokens. The TF–IDF vectorisers of the router use word 1–2-grams (with English stop words removed) and character 3–4-grams. Since a tool's text does not change between requests, its vectors are computed once and cached, so routing a request costs one vectorisation of the request and a few sparse dot products.

E. Machine learning models of the committee

CONCLAVE uses committees of *deliberately heterogeneous* members, because the disagreement of the members is meaningful only if they can disagree for structural reasons [29]. A random forest's trees share an inductive bias, so their disagreement tends to collapse on unfamiliar data; mixing linear, margin-based, generative and tree-based models keeps the disagreement signal informative. All members are implemented with scikit-learn [35] and LightGBM [36].

Logistic regression (LR). A multinomial linear model:

$$p_{\mathrm{LR}}(a_k \mid \mathbf{x}) = \frac{\exp\big(\mathbf{w}_k^\top \mathbf{x} + b_k\big)}{\sum_{j=1}^{K} \exp\big(\mathbf{w}_j^\top \mathbf{x} + b_j\big)}, \tag{5}$$

where $\mathbf{w}_k$ and b_k are the weight vector and bias of action a_k. The parameters minimise the cross-entropy loss plus an L_2 penalty with inverse regularisation strength $C = 4$, using L-BFGS for at most 400 iterations.

Stochastic gradient descent with modified Huber loss (SGD). A one-vs-rest linear model with scores $f_k(\mathbf{x}) = \mathbf{w}_k^\top \mathbf{x} + b_k$, trained by stochastic gradient descent on the loss

$$L(y, f) = \begin{cases} \max(0,\, 1 - yf)^2, & yf \geq -1, \\ -4yf, & \text{otherwise}, \end{cases} \tag{6}$$

with label $y \in \{-1, +1\}$, an L_2 penalty $\alpha = 10^{-5}$ and 25 epochs. Its class probabilities are $p_{\mathrm{SGD}}(a_k \mid \mathbf{x}) \propto \frac{1}{2}\big(\mathrm{clip}(f_k(\mathbf{x}), -1, 1) + 1\big)$, normalised to sum to one.

Naive Bayes (NB). A generative model. For the non-negative TF–IDF features of HumAID we use complement naive Bayes, $\ln p_{\mathrm{NB}}(a_k \mid \mathbf{x}) \propto -\sum_j x_j \ln \bar{\theta}_{kj}$, where $\bar{\theta}_{kj}$ is the smoothed relative frequency of feature j in all classes *except* a_k. The flood features contain negative standardised values, so Bernoulli naive Bayes is used on binarised features $\beta_j = \mathbb{I}[x_j > 0]$:

$$\begin{aligned} \ln p_{\mathrm{NB}}(a_k \mid \mathbf{x}) = {} & \ln \pi_k - \ln Z \\ & + \sum_j \Big[\beta_j \ln \theta_{kj} + (1 - \beta_j) \ln(1 - \theta_{kj})\Big], \end{aligned} \tag{7}$$

where π_k is the prior of action a_k, $\theta_{kj} = (n_{kj} + \alpha)/(n_k + 2\alpha)$ is the smoothed probability that feature j is active for action a_k (n_{kj} active counts among n_k training examples, $\alpha = 0.3$), and Z normalises the probabilities.

LightGBM. A gradient-boosted ensemble of decision trees [36],

$$F_T(\mathbf{x}) = F_0 + \sum_{m=1}^{T} \eta\, f_m(\mathbf{x}), \tag{8}$$

where each tree f_m is grown leaf-wise to fit the negative gradient of the loss, η is the learning rate and T the number of trees ($T = 200$, $\eta = 0.08$ and 31 leaves for the router). The output score is converted to a probability by the logistic function.

Random forest (RF). An average of B decision trees, each trained on a bootstrap sample with random feature subsets [30]:

$$p_{\mathrm{RF}}(a \mid \mathbf{x}) = \frac{1}{B} \sum_{b=1}^{B} p_b(a \mid \mathbf{x}), \tag{9}$$

with $B = 60$ trees of maximum depth 10 and at least four samples per leaf in the router.

Two committees are used. The *decision committee*, used for the flood agent and for HumAID, contains LR, SGD and NB; all three are linear in the features, which makes the committee small and fast. The *ranking committee*, used for tool routing, contains LightGBM, RF and LR.

F. Ensemble aggregation

The members' posteriors are combined by soft voting:

$$\bar{p}(a \mid \mathbf{x}) = \sum_{m=1}^{M} w_m\, p_m(a \mid \mathbf{x}), \qquad w_m \geq 0, \quad \sum_{m=1}^{M} w_m = 1, \tag{10}$$

where p_m is the posterior of member m and w_m its weight. In all reported agents we use uniform weights, $w_m = 1/M$. The framework also supports weights learnt from out-of-fold log-loss,

$$w_m = \frac{\ell_m - \min_i \ell_i + \epsilon}{\sum_{j=1}^{M} \big(\ell_j - \min_i \ell_i + \epsilon\big)}, \qquad \ell_m = -\frac{1}{F} \sum_{f=1}^{F} \mathrm{LL}_m^{(f)}, \tag{11}$$

where $\mathrm{LL}_m^{(f)}$ is the log-loss of member m on fold f of an $F = 3$-fold stratified split and $\epsilon = 10^{-6}$; the effect of this choice is analysed in Section IV-F. When episodic memory is enabled, its prior π_t (Section III-J) is blended with the committee:

$$p_t(a) = (1 - \lambda)\, \bar{p}_t(a) + \lambda\, \pi_t(a), \tag{12}$$

with blending weight $\lambda = 0.25$. The selected tool is $a_t^* = \arg\max_a p_t(a)$.

G. Uncertainty decomposition

For one observation, let $\tilde{p} = \frac{1}{M} \sum_m p_m$ be the plain mean of the member posteriors over the K actions, and let $H[p] = -\sum_{k=1}^{K} p_k \ln p_k$ be the Shannon entropy. Following the standard entropy-based formulation [32], the total uncertainty is split into two parts:

$$U^{\mathrm{tot}} = \frac{H[\tilde{p}]}{\ln K} = U^{\mathrm{ale}} + U^{\mathrm{epi}}, \tag{13}$$

$$U^{\mathrm{ale}} = \frac{1}{M \ln K} \sum_{m=1}^{M} H[p_m], \qquad U^{\mathrm{epi}} = U^{\mathrm{tot}} - U^{\mathrm{ale}}, \tag{14}$$

where division by $\ln K$ scales every term to $[0, 1]$. The aleatoric part U^{ale} is the average entropy of the members: it is high when the members *agree* that the situation is ambiguous, so more sensing will not help and the agent should abstain. The epistemic part U^{epi} is the mutual information between the prediction and the member index, i.e. the members' *disagreement*: it is high when the agent is outside its competence, and it should gather evidence or escalate. Two further statistics are used,

$$c = \max_k \tilde{p}_k, \qquad \delta = \tilde{p}_{(1)} - \tilde{p}_{(2)}, \tag{15}$$

the confidence and the margin, where $\tilde{p}_{(1)}$ and $\tilde{p}_{(2)}$ are the two largest probabilities. A single model cannot separate the two kinds of uncertainty without expensive sampling; a committee obtains them from one forward pass.

H. Uncertainty gate and threshold calibration

The gate maps these statistics to a control decision. The order of the checks encodes a safety policy:

$$g = \begin{cases} \text{gather (or escalate)}, & U^{\text{epi}} > \tau_e, \\ \text{abstain}, & U^{\text{ale}} > \tau_a, \\ \text{gather (or escalate)}, & c < \tau_c \text{ or } \delta < \tau_\delta, \\ \text{act}, & \text{otherwise}, \end{cases} \tag{16}$$

where τ_e, τ_a, τ_c and τ_δ are the epistemic, aleatoric, confidence and margin thresholds. "Gather" is used when the agent has an evidence-gathering action; otherwise the decision is escalated to a human.

The thresholds τ_c and τ_e are not chosen by hand. They are calibrated on a held-out validation set V by selective prediction [33], [34]: among all threshold pairs, we choose the one that accepts the most decisions while the accuracy of the accepted decisions meets a target precision π^*,

$$(\tau_c^*, \tau_e^*) = \arg\max_{\tau_c, \tau_e} \frac{|\mathcal{S}|}{|V|} \quad \text{s.t.} \quad \frac{1}{|\mathcal{S}|} \sum_{i \in \mathcal{S}} \mathbb{I}[\hat{y}_i = y_i] \geq \pi^*, \tag{17}$$

where $\mathcal{S} = \{i \in V : c_i \geq \tau_c,\ U_i^{\text{epi}} \leq \tau_e\}$ is the set of accepted decisions, $\hat{y}_i$ and y_i are the predicted and true labels, and $|\mathcal{S}| \geq 0.05|V|$. The search uses a grid of 24 values of $\tau_c \in [0, 0.95]$ and 24 values of $\tau_e \in [0.02, 0.80]$, with $\pi^* = 0.90$; $\tau_a = 0.60$ and $\tau_\delta = 0.10$ are fixed.

I. Open-vocabulary tool routing and abstention

Instead of classifying a request into a fixed set of tool labels, the router scores *(request, tool) pairs*. For a request q with candidate tools $\mathcal{T}(q)$, the score of tool τ is the mean probability, over the M members of the ranking committee, that τ is the correct tool:

$$\begin{aligned} s(q, \tau) &= \frac{1}{M} \sum_{m=1}^{M} P_m\big(y = 1 \mid \varphi(q, \tau)\big), \\ \tau^* &= \arg\max_{\tau \in \mathcal{T}(q)} s(q, \tau). \end{aligned} \tag{18}$$

Because the router scores pairs, a tool registered after training is scored by the same features as any other, so the agent gains new capabilities without retraining. For the uncertainty decomposition, each member's scores are normalised over the candidates, $\tilde{P}_m(\tau \mid q) = P_m(y = 1 \mid \varphi(q, \tau)) / \sum_{\tau'} P_m(y = 1 \mid \varphi(q, \tau'))$, and Eq. (13) is applied. Abstention — deciding that no candidate tool applies — is a separate binary task. Its ten features summarise the distribution of pair features over the candidates (the maximum, mean, spread and standard deviation of the word similarity, the maximum and mean coverage, the maximum and mean parameter overlap, the number of candidates and the request length), and a committee of a random forest, extra trees and LR predicts whether the request is irrelevant; the agent abstains when the mean probability exceeds 0.5.

J. Episodic memory

An LLM agent adapts to a new situation by placing examples in its prompt. CONCLAVE instead keeps a bounded store of past cases $(\mathbf{k}_i, a_i, \varrho_i)$, where $\mathbf{k}_i$ is a key derived from the feature vector (a fixed random projection to 512 dimensions when the vector is longer), a_i is the action taken and ϱ_i the reward received. For a query key $\mathbf{k}$, the $n = 5$ nearest cases by cosine distance form the set $\mathcal{N}(\mathbf{k})$, and the memory prior is

$$\pi(a \mid \mathbf{k}) \propto \sum_{i \in \mathcal{N}(\mathbf{k}),\ a_i = a,\ \varrho_i > 0} \text{sim}(\mathbf{k}, \mathbf{k}_i)\, \varrho_i, \tag{19}$$

where $\text{sim} = 1 - d_{\cos}$; the prior is uniform when no positive case is found. The store holds at most 2,000 cases; when it is full, the case with the lowest utility $\varrho_i + 0.1\, u_i$ is removed, where u_i is the number of times the case has been retrieved. This is the case-based-reasoning component proposed conceptually for flood early warning [3], at a cost of kilobytes and microseconds.

K. Planner and safety constraints

The committee decides *what* the situation requires; a symbolic planner decides *in which order* to act when a goal needs several tools. Each tool declares preconditions and effects as sets of predicates. The planner performs an A* forward search over states S (sets of true predicates), ordering nodes by

$$f(n) = g(n) + h(n), \qquad h(n) = \big|G \setminus S_n\big|, \tag{20}$$

where $g(n)$ is the total cost of the tools on the path to node n, S_n is its state, G is the set of goal predicates, and $h(n)$ counts the unmet goal predicates. Applying a tool adds its effects, $S' = S \cup \text{eff}(\tau)$. The search is bounded by a depth of 8 and 5,000 expansions, so termination is guaranteed. The heuristic is admissible, and the plan therefore cost-optimal, when every tool adds at most one goal predicate at a cost of at least one; the shipped disaster tools satisfy this condition.

Finally, a constraint layer sits *above* the learned policy. Each constraint is a predicate $C_j(a, o)$ that is checked after the gate and before execution — for example, irreversible tools such as opening a sluice gate require a confidence of at least κ, and a tool may be fired only a limited number of times per episode. If any constraint is violated, the action is converted into an escalation and the reason is recorded; a rule that fails to evaluate is treated as violated. Thus, no statistical model can authorise an irreversible act on its own.

L. Decision loop of the agent

Algorithm 1 and Fig. 2 summarise how the components work together at every time step. Every stage is a bounded amount of arithmetic, so a decision has a fixed worst-case cost, and no stage needs the network.

M. Flood-response simulator

Classification benchmarks cannot show whether a framework behaves well as an *agent*, because they have no sequence of decisions. We therefore built a discrete-time flood-response

Algorithm 1 Decision loop of a CONCLAVE agent

Input: featuriser ϕ; committee $\{p_m, w_m\}_{m=1}^{M}$; thresholds $\tau_c, \tau_\delta, \tau_e, \tau_a$; memory $\mathcal{M}$ (optional); constraints $\{C_j\}$; tool registry.
Output: an audited action a_t at every step t.

1) Receive the observation $o_t = (s_t, r_t)$.
2) Compute the features $\mathbf{x}_t = \phi(o_t)$ by Eqs. (1)–(4).
3) For $m = 1, \dots, M$, compute the member posterior $p_m(\cdot \mid \mathbf{x}_t)$.
4) Aggregate $\bar{p}_t$ by Eq. (10); if $\mathcal{M}$ is not empty, blend it with the memory prior by Eq. (12).
5) Compute U_t^{ale}, U_t^{epi}, c_t and δ_t by Eqs. (13)–(15).
6) Apply the gate of Eq. (16). If the outcome is *act*, set $a_t = \arg\max_a p_t(a)$; otherwise set a_t to the gate outcome.
7) If any constraint $C_j(a_t, o_t)$ is violated, set a_t to *escalate* and record the reason.
8) Execute a_t through the registry; for multi-step goals, order the tools with the planner of Eq. (20).
9) Receive the reward R_t; if a_t is a tool and $R_t > 0$, store the case $(\mathbf{k}_t, a_t, R_t)$ in $\mathcal{M}$.

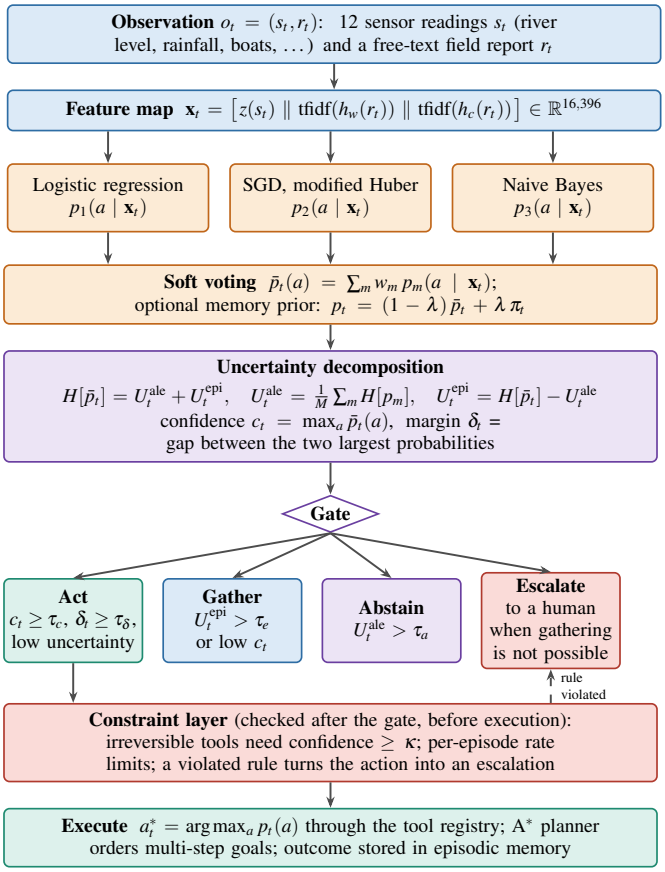


Fig. 2. Decision pipeline of a CONCLAVE agent at one time step. The committee posterior is decomposed into aleatoric and epistemic uncertainty, and the gate chooses one of four outcomes; the constraint layer can still convert an action into an escalation before execution.

environment in which the agent operates a river district for $T = 40$ steps and chooses among six tools: *monitor*, *issue alert*, *evacuate zone*, *deploy rescue*, *request supplies* and *open sluice* (irreversible). Decisions compound: early evacuation costs resources but saves lives if the flood arrives, and false alarms reduce the credibility of later warnings.

Dynamics. Each episode draws an initial river level $\ell_0 \sim \mathcal{U}(1,2)$ m, initial rainfall $\rho_0 \sim \mathcal{U}(0,4)$, a surge time t_s uniformly between step 8 and step $T-9$, and a surge magnitude $S \sim \mathcal{U}(2.5,6)$ m. Rainfall and river level evolve as

$$\rho_{t+1} = \text{clip}\big(\rho_t + \varepsilon_t + 3 \cdot \mathbb{I}[|t - t_s| < 3],\ 0,\ 20\big), \tag{21}$$

$$\ell_{t+1} = \text{clip}\big(\ell_t + 0.11\,\rho_{t+1} - d_t + S \cdot \mathbb{I}[t = t_s],\ 0.4,\ 14\big), \tag{22}$$

where $\varepsilon_t \sim \mathcal{N}(0, 1.2^2)$ and the drainage d_t is 0.45 when the sluice is open and 0.22 otherwise. When the level exceeds 4.5 m, a fraction $0.05 \min\{(\ell_{t+1} - 4.5)/3,\ 1\}$ of the population that has not been evacuated becomes stranded, and 3% of the stranded people become casualties at every step until they are rescued. The hidden flood risk, which the agent never observes, is

$$\varrho_t = \min\big\{1,\ 0.65\,\chi_t + 0.35\,\sigma(\ell_t - 4)\big\}, \tag{23}$$

where $\chi_t = e^{-(t_s - t)/4}$ for $t \leq t_s$ and $\chi_t = 1$ afterwards, and σ is the logistic function.

Observations. The agent sees 12 sensor values — river level, rainfall and level change (with 6% multiplicative noise), elapsed time, available boats and supplies, evacuated fraction, number of stranded people, public credibility, warning and sluice status, and the connectivity flag — and a field report chosen from risk-dependent templates; for example, a risk above 0.75 produces reports such as "urgent people trapped on rooftops need boats immediately".

Reward. An alert issued when $\varrho_t > 0.45$ earns $+3$, otherwise it costs 4 and reduces credibility by 0.15; evacuation earns $0.02\,\varrho_t$ per person moved minus a cost of 2; a rescue earns 0.12 per person saved (an ineffective rescue costs 1); requesting supplies costs 0.5; opening the sluice costs 5 for farm damage and earns 12 if $\varrho_t > 0.6$; monitoring costs 0.05; an escalation costs 0.4 and gathering or abstaining 0.15. Each new casualty costs 0.35, and at the end of the episode the agent receives $+0.005$ per evacuated person, -0.2 per casualty and -0.05 per unit of farm damage. The episode return is $G = \sum_{t=1}^{T} R_t$.

Connectivity. At every step with a working network, an outage starts with probability q and lasts six steps. We use $q \in \{0, 0.1, 0.2, 0.3, 0.4, 0.5\}$, which leaves the network down for 0%, 35.3%, 53.0%, 62.3%, 68.6% and 73.1% of the steps, respectively.

Expert controller. The training labels come from a rule-based expert that sees the true risk: it deploys rescue if more than five people are stranded and boats are available; opens the sluice if $\varrho_t > 0.8$ and the level exceeds 5 m; issues an alert if $\varrho_t > 0.55$ and no warning has been issued; evacuates if $\varrho_t > 0.6$ and less than 70% of the population has left; requests supplies if boats or supplies run low; and otherwise monitors.

N. NumPy-only edge runtime

Training needs scikit-learn and SciPy, but deciding does not. Every member of the decision committee is linear in the features, so a decision needs only the hash function of Eq. (1), the TF–IDF weights of Eq. (2), and a few matrix–vector products. The export step writes the fitted featuriser,

the members' weights and the gate thresholds into a single compressed NumPy file. Hash buckets that never occurred in the training data carry exactly zero weight in LR and SGD and a constant contribution in NB, so only the columns seen in training are stored. The runtime is a single Python file that imports nothing except NumPy and the standard library; it re-implements MurmurHash3, the n-gram analysers, the member posteriors, Eqs. (10)–(15) and the gate. The deployment on a board is therefore two files: the runtime and the exported agent.

O. Compared agents and baselines

Agents for sequential control. All agents act in the same simulator episodes. (i) A *cloud agent* follows the expert controller with access to the true risk; it represents the upper bound of a remote LLM-based agent, since no real language model would act better than the expert that generated the labels. When the network is down it cannot act, and a *cache* variant repeats its last action instead. (ii) *Static threshold rules* act on the gauge (evacuate above 5.5 m, alert above 4.0 m), as a typical fielded system does. (iii) A *random* policy gives the lower bound. (iv) Eight other *machine-learning-based agents* are trained on exactly the same 6,400 decisions and features as CONCLAVE: a case-based-reasoning agent (k-NN with $k = 5$ and cosine distance, as in CBR systems [3]); a single decision tree; Bernoulli naive Bayes; logistic regression; a multilayer perceptron (one hidden layer of 64 ReLU units, Adam, 60 epochs); a random forest (100 trees); LightGBM (200 trees, 63 leaves); and a *hard-vote ensemble* of the multilayer perceptron, the random forest and k-NN, which follows the majority-vote design of ensemble flood-warning frameworks [4].

Baselines for routing and triage. For tool routing, a lexical router selects the tool with the highest word TF–IDF similarity, without any learning, and single-model rankers (LR, RF, LightGBM) are compared with the ranking committee. For HumAID, we compare with the transformer and classical baselines reported by the dataset authors on the same split, and with single LR, SGD and NB classifiers trained on our features.

P. Evaluation metrics

Tool routing is measured by top-1 accuracy, $\mathrm{Acc} = \frac{1}{N}\sum_{i=1}^{N} \mathbb{I}[\hat{\tau}_i = \tau_i]$, and abstention by accuracy, F1 and the area under the ROC curve (AUC). Crisis triage is measured by the weighted and macro F1 scores,

$$\begin{aligned} \mathrm{F1}_k &= \frac{2P_kR_k}{P_k + R_k}, \\ \mathrm{F1}_w &= \sum_{k=1}^{K} \frac{n_k}{N}\,\mathrm{F1}_k, \qquad \mathrm{F1}_m = \frac{1}{K}\sum_{k=1}^{K} \mathrm{F1}_k, \end{aligned} \tag{24}$$

where n_k is the number of test examples of class k, and P_k and R_k are its precision and recall. Sequential control is measured by the mean episode return $\bar{G}$ and the mean number of casualties over 120 episodes. Two agents A and B are compared on the *same* episode seeds by the paired difference

$$\Delta = \frac{1}{n}\sum_{i=1}^{n} \left(G_i^A - G_i^B\right), \tag{25}$$

TABLE IV
TOOL ROUTING ON BFCL V3, TOP-1 ACCURACY (%). LATENCY IS PER SINGLE DECISION, NOT AMORTISED OVER A BATCH. †EVERY TEST TOOL UNSEEN IN TRAINING. ‡IRRELEVANCE DETECTION: ACCURACY SHOWN; AUC 0.921.

Split	n	Lexical	**Ours**	ms
`multiple`	60	90.0	**98.3**	4.72
`live_multiple`	316	79.4	**90.8**	3.99
`live_mult.`†	346	56.7	**78.9**	4.35
Abstention‡	2,628	—	84.3	0.11

whose 95% confidence interval is obtained from 4,000 bootstrap resamples of the episodes; a difference is significant when the interval excludes zero. The quality of uncertainty is measured by the area under the accuracy–coverage curve (AURC) and by the AUROC for separating clean from shifted observations.

Latency is the median and the 99th percentile (p99) of single decisions, taken from the quietest of five trials. Since we do not have the physical boards, on-device latency and energy are estimated from the host measurement,

$$\hat{t}_{\mathrm{dev}} = \frac{t_{\mathrm{host}}}{\rho_{\mathrm{dev}}}, \qquad \hat{E}_{\mathrm{dev}} = \hat{t}_{\mathrm{dev}}\, P_{\mathrm{dev}}, \tag{26}$$

where ρ_{dev} and P_{dev} are the relative throughput and the thermal design power of the board (Table III). Memory is the peak resident set size of a fresh process that loads the agent and makes several hundred decisions, and model size is the size of the compressed serialised file.

Q. Implementation details

All experiments run on a 2-core x86-64 Linux container with 3.82 GB of RAM, which is itself close to a Raspberry Pi profile. The software is Python 3.10, scikit-learn 1.7.2 [35], LightGBM 4.7.0 [36] and NumPy 2.2.6. The random seed is fixed at 42, and every result file records the platform, seed and code version. Two defaults are set from measurement. First, all models predict with a single thread: an agent decides about one observation at a time, and creating a thread pool costs about 20× more than the prediction itself at that batch size (90 ms against 4 ms). Second, the router uses shallow forests: depth-10, 60-tree forests lose well under one point of accuracy compared with unbounded 200-tree forests, while the serialised model becomes about 9× smaller.

IV. RESULTS

This section presents the results in the order of the test beds in Table II: tool routing, crisis triage, sequential control under connectivity loss and resource cost. These are followed by the comparison with other machine-learning-based agents, the ablation study of the committee, and the analysis of the uncertainty signal.

A. Tool routing and abstention

Table IV and Fig. 3 show the routing results. The ranking committee reaches 98.3% top-1 accuracy on the standard `multiple` split and 90.8% on the user-contributed

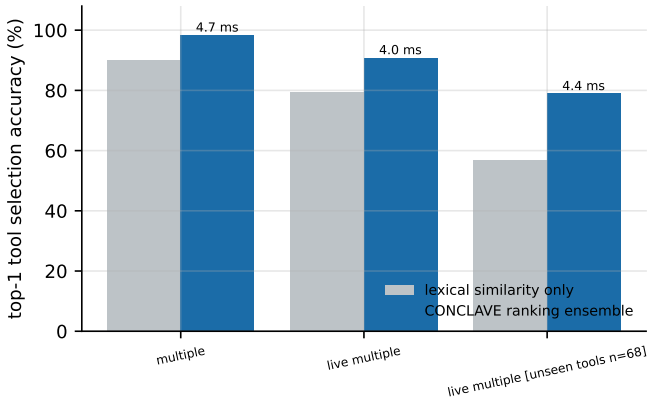


Fig. 3. Tool routing on BFCL v3. Learning adds 8 to 22 points over lexical matching, and the largest gain is obtained where the tools were unseen in training.

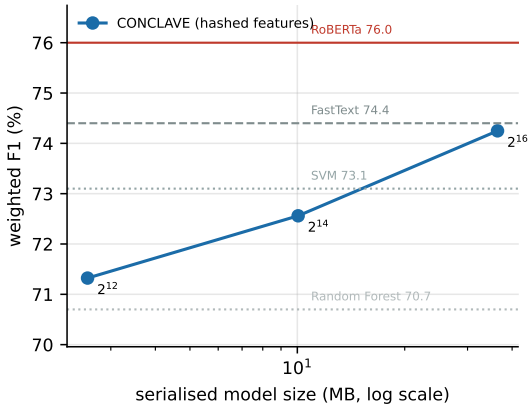


Fig. 4. Weighted F1 against model size on HumAID. Each point is one number of hash buckets; horizontal lines are the baselines reported by the dataset authors.

TABLE V
HUMAID TRIAGE ON THE COMBINED 10-CLASS SPLIT. BASELINES ARE AS REPORTED BY THE HUMAID AUTHORS (TABLE 7 OF THEIR PAPER, ROW "ALL"). THE NUMBER OF HASH BUCKETS IS CONCLAVE'S DEPLOYMENT PARAMETER.

Model	Weighted F1	Size	ms/dec.
Reported by the dataset authors			
RoBERTa	**0.760**	∼500 MB	—
BERT, DistilBERT, XLM-R	0.758	≤500 MB	—
FastText	0.744	—	—
SVM	0.731	—	—
Random Forest	0.707	—	—
This work			
CONCLAVE 2^{16}	0.743	36.4 MB	4.94
CONCLAVE 2^{14}	0.726	10.1 MB	1.82
CONCLAVE 2^{12}	0.713	**2.6 MB**	**1.27**

TABLE VI
MEAN EPISODE RETURN UNDER CONNECTIVITY LOSS, 120 EPISODES PER ROW. Δ IS THE PAIRED DIFFERENCE (CONCLAVE MINUS CLOUD AGENT) OVER IDENTICAL EPISODE SEEDS, WITH ITS BOOTSTRAP 95% CONFIDENCE INTERVAL.

Offline	**CONCLAVE**	Cloud agent	Δ	95% CI
0.0%	36.5	35.8	+0.6	[−0.7, 1.9]
35.3%	33.9	−34.2	+68.1	[57.6, 78.9]
53.0%	34.3	−77.7	+112.0	[100.6, 123.6]
62.3%	34.8	−99.3	+134.2	[122.9, 145.8]
68.6%	34.9	−113.2	+148.1	[136.2, 160.3]
73.1%	33.5	−127.3	+160.8	[148.5, 173.0]

`live_multiple` split, at 3.99 ms per decision (p99 7.09 ms) with a 631 KB model. The lexical-similarity router shows how much of this comes from learning: the committee is 11.4 points better on `live_multiple` and 22.2 points better on the unseen-tool split, where it still selects the correct tool for 78.9% of requests although none of these tools was seen in training. For abstention, i.e. deciding that no candidate tool applies, the committee reaches 84.3% accuracy, an F1 score of 0.815 and an AUC of 0.921 in 0.11 ms. We do not compare these numbers with language-model scores on the BFCL leaderboard, because its headline metric also checks the arguments of each call, whereas the CONCLAVE router selects the tool; the cost of the agent is reported in Section IV-D.

B. Crisis triage on HumAID

Table V and Fig. 4 give the trade-off between accuracy and model size on the official combined split. With 2^{16} buckets, CONCLAVE reaches a weighted F1 of 0.743 (macro F1 0.716), which is 1.8 points below RoBERTa with a 14× smaller model; it matches FastText (0.743 against 0.744) and clearly exceeds the authors' SVM (0.731) and random forest (0.707). Even the 2.6 MB configuration (0.713) is better than their random forest, at a size where a transformer cannot be deployed at all. The calibrated gate of Section III-H is also effective on this task: on the validation split, the thresholds $\tau_c = 0.661$ and $\tau_e = 0.223$ accept 54.0% of the tweets with a precision of 90.5%, so the remaining tweets can be passed to a human analyst. Readers who compare with this dataset should note that Table 7 of the HumAID paper also contains a column average of 0.781, which is computed across per-event and event-type rows; it is not the result for the combined split and should not be used as the baseline for it.

C. Sequential control under connectivity loss

This experiment tests the central claim of the paper, and it is designed to be unfavourable to CONCLAVE. The cloud agent follows the *expert* controller, which sees the hidden flood risk, while CONCLAVE sees only the noisy sensors and the text report and was trained by behaviour cloning on the same expert. CONCLAVE is therefore weaker in capability, and any advantage must come from availability. In this experiment the gate is not allowed to withhold actions, so that the comparison measures the policy itself; the use of uncertainty for escalation is analysed in Section IV-G.

Table VI and Fig. 5 show the result. With full connectivity, the two agents are statistically indistinguishable ($\Delta = 0.6$, CI $[-0.7, 1.9]$), which confirms that behaviour cloning has recovered the expert policy and that the comparison is fair.

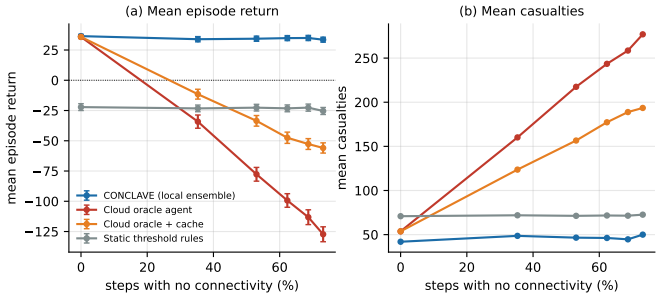


Fig. 5. Return and casualties against the fraction of offline steps. The cloud agent follows the expert policy; its collapse is caused by lost availability, not by lack of capability.

TABLE VII
DECISION COST OF THE SCIKIT-LEARN AGENT UNDER EMULATED BOARD PROFILES. MEASURED VALUES WERE OBTAINED WITH THE BOARD'S CORE COUNT AND, FOR BOARDS WITH AT LEAST 1 GB, AN ENFORCED RAM CEILING; ON-DEVICE COLUMNS ARE ESTIMATED BY EQ. (26).

	Measured (host)		Estimated on-device	
Board	mean	p99	mean	energy
Host (2-core x86)	2.47 ms	3.07 ms	2.47 ms	37.1 mJ
Jetson Nano	2.46 ms	3.23 ms	9.47 ms	94.7 mJ
Raspberry Pi 4B	2.44 ms	3.64 ms	8.72 ms	55.8 mJ
Raspberry Pi 3B+	2.43 ms	2.87 ms	14.27 ms	71.4 mJ
Pi Zero 2 W	2.49 ms	3.37 ms	22.59 ms	56.5 mJ

From the first outage setting onwards, CONCLAVE is significantly better, and the gap increases monotonically with the fraction of offline steps. In terms of casualties, the mean of the cloud agent rises from 53.7 to 276.9 as connectivity degrades, whereas CONCLAVE stays between 41.9 and 50.0. A last-action cache recovers only part of the loss (-55.9 at 73% offline), because repeating a stale action in a fast-changing flood is only slightly better than doing nothing. Static threshold rules, which represent many fielded systems, are not affected by outages, but they are much worse than both agents throughout (-22.2 to -25.4).

D. Resource cost on low-cost hardware

The complete disaster agent — featuriser and committee — serialises to 232 KB. Table VII shows that, with the core counts of the boards applied, a decision takes about 2.5 ms on the host, and the estimated mean latency is 22.6 ms (p99 30.6 ms) even on a Raspberry Pi Zero 2 W, with an energy of about 56 mJ per decision. With the scikit-learn stack, however, the process needs about 185 MB of memory, most of which is taken by SciPy and scikit-learn rather than by the agent.

The NumPy-only runtime of Section III-N removes this overhead (Table VIII and Fig. 6). On 2,400 held-out observations it reproduces the scikit-learn agent exactly: all actions and all gate decisions are identical, and the largest difference in any posterior probability is 8×10^{-14}. The exported file keeps 2,486 of the 16,396 feature columns and occupies 113 KB. Peak memory falls from 185.7 MB to 32.2 MB, and the runtime runs under an enforced address-space limit of 256 MB, half the RAM of a Pi Zero 2 W. The median decision

TABLE VIII
THE SAME TRAINED FLOOD AGENT DEPLOYED WITH SCIKIT-LEARN AND WITH THE NUMPY-ONLY EDGE RUNTIME. BOTH RUN IN A FRESH PROCESS ON 600 HELD-OUT OBSERVATIONS.

Quantity	scikit-learn	**NumPy runtime**
Libraries on the device	sklearn, SciPy	NumPy only
Model file	232 KB	**113 KB**
Peak RAM	185.7 MB	**32.2 MB**
Memory cap passed	512 MB	**256 MB**
Median latency (host)	2.72 ms	**0.19 ms**
p99 latency (host)	4.18 ms	**0.30 ms**
Est. median, Pi Zero 2 W	24.7 ms	**1.71 ms**
Identical decisions	—	2,400 / 2,400

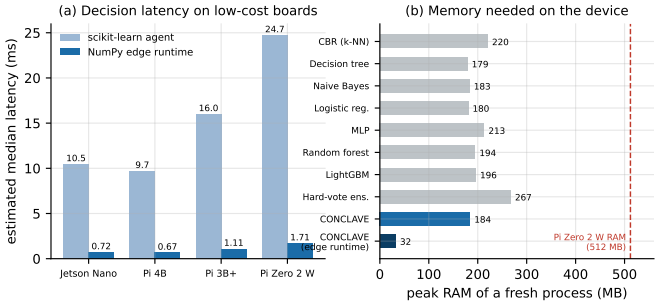


Fig. 6. (a) Estimated median decision latency of the flood agent on each board with the scikit-learn stack and with the NumPy edge runtime. (b) Peak RAM of a fresh process for every ML agent of Section IV-E and for CONCLAVE with the edge runtime; the dashed line is the RAM of a Raspberry Pi Zero 2 W.

time falls from 2.72 ms to 0.19 ms, because the overhead of the general-purpose library calls disappears; the estimated median on a Pi Zero 2 W is 1.71 ms, and 0.67 ms on a Raspberry Pi 4B. The closed-loop agent of Section IV-C was also exported and run inside the simulator through the runtime: its returns are identical to those of the scikit-learn agent (36.5 and 33.5), with 31.4 MB of memory.

E. Comparison with other machine-learning-based agents

To show the contribution of CONCLAVE more directly, we compare it with eight other agents whose decision cores are machine learning models, and with the reference agents of Section IV-C. All ML agents run fully offline on the same CPU and are trained on the same 6,400 decisions with the same features, so they differ only in the decision core. They are evaluated in the two conditions that matter most in the field: *no internet* (the network down for 73% of the steps) and *low-cost hardware* (latency, model size and memory, and the estimated latency on a Raspberry Pi Zero 2 W). Table IX and Fig. 7 summarise the results.

No internet. The cloud agent loses most of its value when the network fails: its return falls from 35.8 to -127.3 and its casualties rise to 276.9. All ML agents are unaffected by the outage because they decide locally, but their quality differs widely. CONCLAVE obtains the highest return of all agents, both with the network up (36.5) and with 73% of the steps offline (33.5), and the fewest casualties (41.9 and 50.0). In the paired comparison over identical episodes, CONCLAVE

TABLE IX
COMPARISON OF CONCLAVE WITH OTHER AGENTS IN THE FLOOD-RESPONSE TASK UNDER NO-INTERNET AND LOW-COST-HARDWARE CONDITIONS. ONE-STEP ACCURACY IS MEASURED ON 2,400 HELD-OUT EXPERT DECISIONS; RETURNS AND CASUALTIES ARE MEANS OVER 120 EPISODES. LATENCY IS THE MEDIAN SINGLE DECISION ON THE HOST; ON-DEVICE LATENCY IS ESTIMATED BY EQ. (26). PEAK RAM IS THAT OF A FRESH PROCESS. THE BEST VALUE AMONG THE ML AGENTS IS IN BOLD.

			Return		Casualties	Latency (ms)		Model	Peak	Uncertainty
Agent	Internet	Acc. (%)	0% off	73% off	73% off	host	Pi Zero	(KB)	RAM (MB)	signal
Reference agents										
Cloud agent (expert policy)	needed	—	35.8	−127.3	276.9	—	—	remote	—	none
Cloud agent + action cache	needed	—	35.8	−55.9	193.5	—	—	remote	—	none
Static threshold rules	no	—	−22.2	−25.4	72.6	<0.01	<0.1	—	—	none
Machine-learning-based agents (all offline)										
CBR (k-NN)	no	93.9	17.1	16.3	81.8	11.84	107.6	2,361	220	confidence
Decision tree	no	**98.7**	34.0	31.6	62.7	1.03	9.3	**16**	179	confidence
Naive Bayes	no	74.4	−57.2	−69.0	190.7	2.06	18.8	73	183	confidence
Logistic regression	no	98.4	33.0	29.7	68.7	1.09	9.9	117	180	confidence
MLP	no	97.0	31.5	29.0	67.6	1.04	9.4	32,575	213	confidence
Random forest	no	91.5	21.9	21.9	69.3	4.30	39.1	1,878	194	confidence
LightGBM	no	98.6	33.4	31.4	64.7	1.40	12.7	1,751	196	confidence
Hard-vote ensemble [4]	no	96.5	27.7	24.5	72.5	15.37	139.8	36,793	267	vote split
CONCLAVE (scikit-learn)	no	96.3	**36.5**	**33.5**	**50.0**	2.45	22.3	249	184	aleatoric + epistemic
CONCLAVE (NumPy runtime)	no	96.3	**36.5**	**33.5**	**50.0**	**0.19**	**1.7**	128	**31**	aleatoric + epistemic

is significantly better than every other ML agent with the network up; with 73% of the steps offline it is significantly better than all of them except the single decision tree ($\Delta = 1.9$, CI $[-0.2, 3.9]$). The decision tree, LightGBM and logistic regression form a second group (returns of 29.7 to 31.6 offline); the multilayer perceptron, the hard-vote ensemble, the random forest and the case-based-reasoning agent follow, and naive Bayes alone fails (-69.0).

An important observation is that one-step accuracy does not predict closed-loop quality. The decision tree (98.7%), LightGBM (98.6%) and logistic regression (98.4%) imitate the expert more accurately than CONCLAVE (96.3%), but they obtain lower returns and, with the network down, 13 to 19 more casualties per episode. In a sequential task, an error changes the next observation, and errors compound. A committee of members with different inductive biases makes errors that are less correlated with each other, so its policy drifts less from situations seen in training. The hard-vote ensemble does not show this benefit (24.5), because majority voting over three members discards the probability information that soft voting keeps.

Low-cost hardware. CONCLAVE is also one of the cheapest agents to run. With scikit-learn, it takes 2.45 ms per decision (22.3 ms estimated on a Pi Zero 2 W), its model is 249 KB and the process needs 184 MB. The single linear and tree models are smaller or faster, but less effective in closed loop, while the case-based-reasoning agent, the random forest and the hard-vote ensemble are 2 to 6× slower and 7 to 150× larger, because they store their training cases or hundreds of trees. With the NumPy runtime, CONCLAVE needs only 0.19 ms per decision (1.69 ms estimated on a Pi Zero 2 W), a 128 KB file and 31 MB of memory, with identical returns; this runtime applies to any committee of linear members, whereas the tree-based, neural and case-based agents need their full libraries on the device. Finally, CONCLAVE is the only agent with both an aleatoric and an epistemic uncertainty estimate, which the gate uses to decide when to escalate; single models provide only their confidence, and the hard-vote ensemble only the split of its votes.

TABLE X
SINGLE MODELS AGAINST THE CONCLAVE COMMITTEE ON ONE-STEP TASKS: TOP-1 TOOL-ROUTING ACCURACY (%) ON THE THREE BFCL SPLITS AND WEIGHTED F1 (%) ON HUMAID (2^{16} BUCKETS). IDENTICAL FEATURES AND SPLITS; — MEANS THE MODEL IS NOT A MEMBER OF THAT COMMITTEE.

Model	`multiple`	`live`	unseen	HumAID
Logistic regression	**98.3**	86.7	**84.1**	**74.7**
Random forest	96.7	88.9	74.9	—
LightGBM	95.0	87.3	65.6	—
SGD (modified Huber)	—	—	—	72.7
Naive Bayes	—	—	—	66.2
CONCLAVE committee	**98.3**	**90.8**	78.9	74.3

Table X completes the comparison for the one-step tasks. In tool routing, the committee is the best ranker on the user-contributed split (90.8%, 1.9 to 4.1 points above any single member) and equals the best one on the standard split (98.3%). On tools never seen in training, logistic regression alone generalises best (84.1%): the tree members learn patterns that are specific to the training tools, and averaging them with the linear member costs 5.2 points on new tools. In crisis triage, logistic regression alone is slightly above the committee (74.7% against 74.3%), while SGD and naive Bayes are lower. Therefore, in one-step classification the committee performs at the level of its best member, and its main advantages are the uncertainty estimate and the closed-loop robustness shown above. For open-vocabulary routing, these results also suggest a practical rule: when most tools will be added after deployment, the linear member should receive a larger weight.

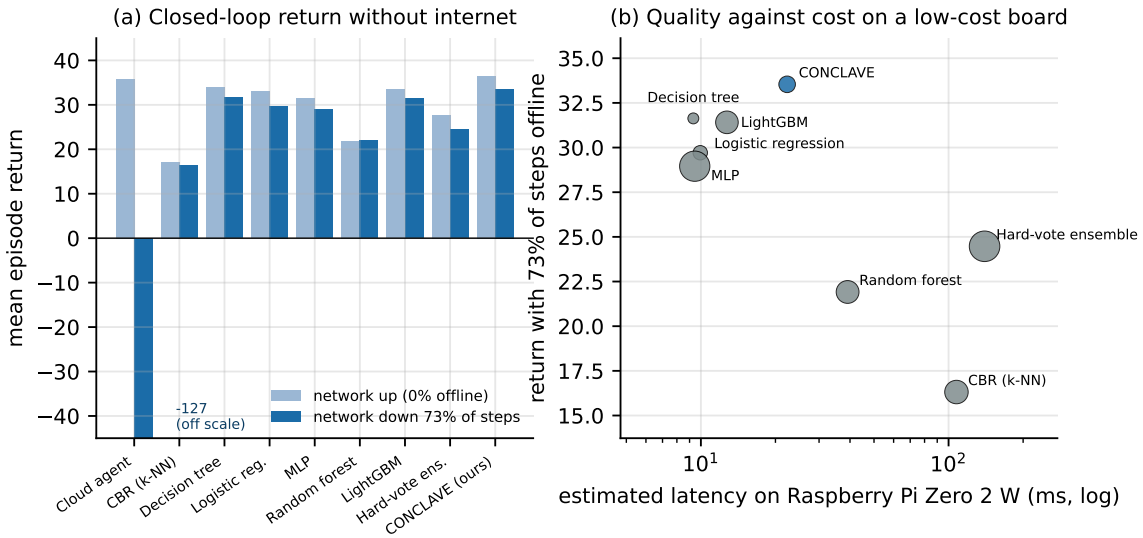


Fig. 7. (a) Mean episode return of every agent with the network up and with the network down for 73% of the steps (naive Bayes, at −57 and −69, is omitted). (b) Return with 73% of the steps offline against the estimated decision latency on a Raspberry Pi Zero 2 W; marker area grows with model size. CONCLAVE gives the best quality at a latency well within real-time limits.

TABLE XI
ABLATION ON THE FLOOD DECISION DATA (ONE-STEP ACCURACY). IDENTICAL FEATURES, SPLITS AND SEEDS; THE ONLY VARIABLE IS THE SET OF MEMBERS. SINGLE MODELS HAVE NO EPISTEMIC TERM BY CONSTRUCTION. THE SHIFT CONDITION IS 60% SENSOR DROPOUT.

			shift AUROC		
Model	Acc. (%)	AURC	conf.	epist.	MB
LogReg only	98.31	0.949	0.662	—	0.08
Random forest only	93.25	0.942	0.577	—	2.45
Extra trees only	92.25	0.940	0.644	—	3.36
HistGB only	98.62	0.950	0.783	—	1.03
LightGBM only	98.75	0.950	0.808	—	1.49
Committee: all trees	91.12	0.932	0.531	0.511	1.43
Committee: default	95.00	0.946	0.753	0.685	0.16
Committee: diverse	98.75	0.949	0.787	0.801	4.05

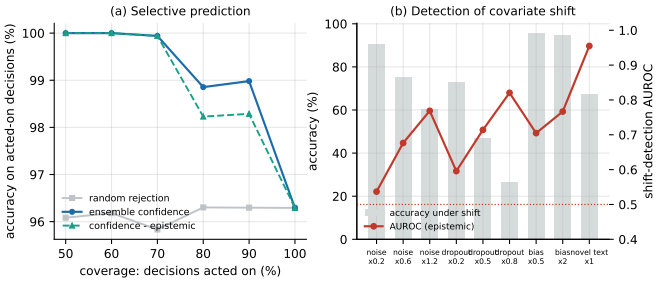


Fig. 8. (a) Selective prediction: accuracy on the decisions the agent acts on, as the fraction of accepted decisions falls. (b) Accuracy under different covariate shifts (bars) and the AUROC of epistemic uncertainty for detecting each shift (line).

F. Ablation study: role of the committee

Table XI isolates the effect of the committee's composition on one-step decisions, using the same features, splits and seeds for every candidate. Three observations can be made.

Accuracy. The best single model (LightGBM, 98.75%) equals the best committee (LightGBM, logistic regression and random forest, 98.75%) in one-step accuracy and AURC. One-step accuracy alone is therefore not the reason to use a committee; the reasons are the closed-loop robustness shown in Section IV-E and the uncertainty signal.

Diversity. The diversity of the members is essential. A committee of three tree models has almost no disagreement (mean epistemic uncertainty 0.007), and its shift-detection AUROC is 0.511, close to chance, while the mixed committee reaches 0.030 and 0.801. Homogeneous members agree even when they are wrong, which is the failure mode anticipated in Section III-E.

Weighting. Under uniform voting, the default committee reaches 95.0% one-step accuracy in this setting, below its best member (98.3%), because the naive Bayes member is much weaker than the other two. Out-of-fold log-loss weighting (Eq. (11)) gives this member almost zero weight and raises the accuracy of the closed-loop agent from 96.3% to 98.0%. In closed loop, however, the log-loss-weighted agent obtains a lower return than the uniform one (33.5 against 36.5 with the network up, and 30.9 against 33.5 with 73% offline; both differences significant) and more casualties (56.9 against 41.9). The weak member still decorrelates the committee's errors, and this matters more in a sequential task than its one-step accuracy suggests. For this reason, uniform voting is used for the closed-loop agent.

G. Analysis of the uncertainty signal

Selective prediction. When the agent is allowed to decline the decisions it is least sure about, the accuracy of the decisions it acts on rises from 96.3% at full coverage to 100% at 50% coverage, whereas random rejection stays at about 96% (Fig. 8a; AURC 0.946). The committee's confidence is therefore a reliable ranking of its own errors.

Detection of degraded inputs. Across sensor noise, sensor dropout, sensor bias and an unfamiliar report vocabulary, uncertainty rises as accuracy falls (Fig. 8b). Epistemic uncertainty detects a novel report vocabulary with an AUROC of 0.955 (accuracy falls from 96.3% to 67.2%) and 80% sensor dropout with an AUROC of 0.820 (accuracy 26.6%). In these tests, the maximum posterior confidence performs at the same level as epistemic uncertainty (for example, 0.960 against 0.955 for the novel vocabulary), so both can be used as the trigger for escalation. Shifts that change only the hidden flood severity are not detected (AUROC ≈ 0.50), because the expert keeps the observations within the training range; detection requires a change in the inputs themselves.

Allocation of human oversight. We also tested whether uncertainty can decide *when* to spend a limited budget of human interventions in a sequential task with 60% sensor dropout. With a budget of 10 interventions per episode, random allocation raised the mean return from -14.7 to $+2.9$, whereas allocation by epistemic uncertainty reached -11.1 and by confidence -10.3. Uncertainty-triggered interventions cluster early in the episode, when uncertainty is high but the consequences are still small, while random allocation spreads them over the whole horizon. This shows that, for sequential oversight, the uncertainty of a decision must be combined with the consequence of an error at that step; a value-of-information criterion of this kind is a direction for future work, while the uncertainty signal remains effective for selective prediction and for detecting degraded inputs.

V. Discussion

A. Why the committee acts better in closed loop

The most important finding of the comparison is that the ranking of agents by one-step accuracy is not the ranking by closed-loop return. Three single models imitate the expert more accurately than CONCLAVE, yet CONCLAVE obtains the highest return and the fewest casualties, with or without the network. In behaviour cloning, every error moves the agent towards situations that are less represented in the training data, where further errors become more likely. The members of CONCLAVE — a discriminative linear model, a margin-based linear model and a generative model — make errors in different situations, so soft voting removes many of them before they can compound. The ablation supports this explanation: removing the influence of the weakest member by log-loss weighting raises one-step accuracy but lowers the closed-loop return, and a committee of similar tree models loses the disagreement signal altogether. For agent design, this suggests that members should be selected for the diversity of their errors, not only for their individual accuracy.

B. Practical implications for disaster response

For flood-prone regions such as the river basins of Bangladesh, the results indicate a practical deployment path. A complete agent that triages field reports, selects response tools and recommends actions fits in about $128\,\mathrm{KB}$, needs about $31\,\mathrm{MB}$ of memory and an estimated $2\,\mathrm{ms}$ per decision on a Raspberry Pi Zero $2\,\mathrm{W}$, a board that can run from a small battery or solar panel. Such an agent continues to work when the mobile network fails, which is exactly the period in which the cloud agent of Section IV-C loses most of its value. Because the output space is a fixed set of registered tools, the agent cannot produce an action outside that set, and the constraint layer keeps irreversible actions — such as opening a sluice gate or ordering an evacuation — under human authority. The calibrated gate adds a measured basis for this hand-over: on HumAID, more than half of the incoming messages can be triaged automatically at 90% precision, and the rest are passed to a human analyst.

C. Complement to language-model agents

CONCLAVE does not replace language models in every task. A language model remains necessary when the action space is open-ended, when the task arrives in unconstrained natural language, or when the agent must generate text, for example reports from imagery and witness accounts [5], [25], scenario-driven collective decision-making [9] or population-scale behavioural simulation [22]. The two kinds of core can work together. ResQConnect already pairs an online agentic layer with an offline tier [15], and Sen et al. place a classical ensemble beneath an LLM [4]. CONCLAVE is suitable for the lowest tier of such a system: the part that must keep deciding when every other tier is unreachable, on whatever hardware survives.

D. Where the cost of an agent comes from

Our results also show how much of an agentic pipeline does not require language modelling. Tool routing is a ranking problem, abstention is binary classification, crisis triage is multi-class classification, and flood response is policy learning. A hashed featuriser and a small committee of classical models solve these problems at a small fraction of the usual cost, and a NumPy-only runtime brings the memory requirement down to the size of the Python interpreter. Operational cost is now being reported alongside accuracy in this literature [24]; we recommend that it becomes a standard part of the evaluation, since it often decides whether a system can be deployed in the field.

VI. Limitations and Future Work

Although the experiments cover routing, triage, sequential control and resource cost, three limitations remain, and each of them defines a direction for future work.

Hardware measurements. The core counts and memory limits of the boards were enforced, and the edge runtime was shown to run within an enforced $256\,\mathrm{MB}$ limit, but on-device latency and energy were estimated from host measurements with published throughput ratios. Measurements on physical Raspberry Pi and Jetson boards are planned as the next step.

Simulated sequential environment. The flood-response environment has hand-specified dynamics, and its expert controller is a rule-based policy rather than an optimal one. The availability result does not depend on these details, but absolute returns are specific to the simulator. Calibrating the simulator

with historical river-gauge and response records from flood-prone districts is an important extension.

Lexical text representation. Hashed n-grams capture spelling and local word order but not compositional meaning, so negation and paraphrase are harder for CONCLAVE than for a transformer, which explains part of the gap on HumAID. In addition, the router selects the tool, while the arguments of a call are supplied by the agent's state, as in the flood agent; extracting typed arguments from free text is outside the present scope. Compact sentence embeddings and schema-guided argument extraction will be studied in future work.

VII. Conclusion

This paper has asked whether an AI agent for disaster response needs a language model at its centre, and has proposed CONCLAVE, an agentic AI framework in which a heterogeneous committee of classical machine learning models serves as the cognitive core. The framework combines hashed text and sensor features, soft voting, an aleatoric–epistemic uncertainty decomposition, a calibrated gate for acting, gathering, abstaining and escalating, case-based memory, symbolic planning, safety constraints and a NumPy-only edge runtime. The experimental results show that CONCLAVE selects the correct tool with 90.8% accuracy in 4 ms, triages crisis tweets within 1.8 F1 points of RoBERTa with a $14\times$ smaller model, and keeps its performance when 73% of the steps are offline, while a cloud-dependent agent collapses from 35.8 to -127.3. Compared with eight other machine-learning-based agents, it achieves the highest closed-loop return and the fewest casualties, and with the edge runtime it needs only 31 MB of memory and an estimated 1.7 ms per decision on a Raspberry Pi Zero 2 W. We conclude that, for bounded and recurring decisions such as those of flood response, "agentic" describes a control architecture rather than a model class, and that a committee of classical models can provide reliable, uncertainty-aware autonomy without the internet and on inexpensive hardware.

Reproducibility and Data Availability

All code, seeds, result files and figure-generation scripts are released under the Apache-2.0 licence. The two public datasets are downloaded from their original sources by `scripts/fetch_data.py`. Every result in this paper is regenerated by eight scripts in `experiments/`; each writes a JSON file that records the platform, seed and code version, and a test suite checks every number quoted in the paper against those files. The complete set of experiments takes about one hour on the 2-core machine described in Section III-Q. The exported edge agent and its runtime are provided in `deploy/` and `conclave/edge/runtime.py`.

References

[1] E. Alotaibi and N. Nassif, "Artificial intelligence in environmental monitoring: in-depth analysis," *Discover Artificial Intelligence*, vol. 4, no. 1, p. 84, 2024.

[2] S. Farazmehr and Y. Wu, "Locating and deploying essential goods and equipment in disasters using AI-enabled approaches: A systematic literature review," *Progress in Disaster Science*, vol. 19, p. 100292, 2023.

[3] N. A. M. Rashid, Z. Z. Abidin, and Z. A. Abas, "Integrating multi-agent system and case-based reasoning for flood early warning and response system," *International Journal of Advanced Computer Science and Applications*, vol. 15, no. 12, 2024.

[4] A. A. A. Sen, F. H. Aljohani, N. M. Bahbouh, A. B. Mnaouer, O. Tayan, and A. B. Alkhodre, "A context-aware flood warning framework integrating ensemble learning and LLMs," *GeoHazards*, vol. 7, no. 1, p. 35, 2026.

[5] Z. Chen, E. Asadi Shamsabadi, S. Jiang, L. Shen, and D. Dias-da Costa, "Integration of large vision language models for efficient post-disaster damage assessment and reporting," *Nature Communications*, vol. 17, no. 1, p. 1481, 2026.

[6] S. Wang, R. Li, H. Wu, J. Li, and Y. Shen, "Fine-grained flood disaster information extraction incorporating multiple semantic features," *International Journal of Digital Earth*, vol. 18, no. 1, p. 2448221, 2025.

[7] Y. Yang, Q. Wang, W. Li, S. Wang, S. Fang, K. Sun, S. Wang, H. Wang, X. Dai, Z. Li, and Y. Zhu, "Automated extraction of spatiotemporal disaster knowledge for urban floods: a multimodal framework based on LLMs and agent," *International Journal of Digital Earth*, vol. 19, no. 1, p. 2640706, 2026.

[8] M. Chen, Z. Tao, W. Tang, T. Qin, R. Yang, and C. Zhu, "Enhancing emergency decision-making with knowledge graphs and large language models," *International Journal of Disaster Risk Reduction*, vol. 113, p. 104804, 2024.

[9] Y. Wang, C. Wang, X. Zhang, and L. Zeng, "Towards intelligent emergency management: A scenario–learning–decision framework enabled by large language models," *Mathematics*, vol. 13, no. 21, p. 3463, 2025.

[10] F. Xu, J. Ma, N. Li, and J. C. Cheng, "Large language model applications in disaster management: An interdisciplinary review," *International Journal of Disaster Risk Reduction*, vol. 127, p. 105642, 2025.

[11] K. Saengtabtim, N. Leelawat, R. Aumnoysombat, M. Adelifar, N. Saengwongwattana, G. Suktavornprasit, A. Suppasri, F. Imamura, and J. Tang, "Harnessing generative AI for enhanced disaster management: a systematic review," *Big Earth Data*, vol. 9, no. 4, pp. 651–675, 2025.

[12] T. Schick, J. Dwivedi-Yu, R. Dessì, R. Raileanu, M. Lomeli, E. Hambro, L. Zettlemoyer, N. Cancedda, and T. Scialom, "Toolformer: Language models can teach themselves to use tools," in *Advances in Neural Information Processing Systems*, vol. 36, 2023, pp. 68 539–68 551.

[13] S. Yao, D. Yu, J. Zhao, I. Shafran, T. L. Griffiths, Y. Cao, and K. Narasimhan, "Tree of thoughts: Deliberate problem solving with large language models," in *Advances in Neural Information Processing Systems*, vol. 36, 2023.

[14] Y. Zhang, R. Sun, Y. Chen, T. Pfister, R. Zhang, and S. Ö. Arik, "Chain of agents: Large language models collaborating on long-context tasks," in *Advances in Neural Information Processing Systems*, vol. 37, 2024.

[15] S. Aththanayake, C. Mallikarachchi, J. Wickramasinghe, S. Kugarajah, D. Meedeniya, and B. Pradhan, "ResQConnect: An AI-Powered multi-agentic platform for human-centered and resilient disaster response," *Sustainability*, vol. 18, no. 2, p. 1014, 2026.

[16] T.-H. Ou, T.-H. Yang, and P.-Z. Chang, "Combination of large language models and portable flood sensors for community flood response: A preliminary study," *Water*, vol. 17, no. 7, p. 1055, 2025.

[17] F. Wang, Z. Zhang, X. Zhang, Z. Wu, T. Mo, Q. Lu, W. Wang, R. Li, J. Xu, X. Tang, Q. He, Y. Ma, M. Huang, and S. Wang, "A comprehensive survey of small language models in the era of large language models: Techniques, enhancements, applications, collaboration with LLMs, and trustworthiness," *ACM Transactions on Intelligent Systems and Technology*, vol. 16, no. 6, pp. 1–87, 2025.

[18] B. Ticona and A. Anastasopoulos, "LLM-Powered automatic translation and urgency in crisis scenarios," in *Proceedings of the 23rd International ISCRAM Conference*, The Hague, the Netherlands, 2026.

[19] J. Ativo, B. Balasubramaniyam, A. Tran, K. Gupta, H. Li, D. Caragea, and C. Caragea, "LLM-guided semi-supervised approaches for social media crisis data classification," in *Proceedings of the 23rd International ISCRAM Conference*, The Hague, the Netherlands, 2026.

[20] B.-J. Park, S.-J. Yong, H.-S. Hwang, and I.-Y. Moon, "Optimizing agent behavior in the MiniGrid environment using reinforcement learning based on large language models," *Applied Sciences*, vol. 15, no. 4, p. 1860, 2025.

[21] K. Valmeekam, M. Marquez, S. Sreedharan, and S. Kambhampati, "On the planning abilities of large language models – a critical investigation," in *Advances in Neural Information Processing Systems*, vol. 36, 2023.

[22] R. Sultimov, M. Mozikov, D. Abramov, M. Kovalchuk, M. Malykh, I. Makarov, A. Osiptsov, A. Volkov, and Y. Maximov, "RESPOND: Realistic environment simulation of population and natural disasters with LLM-Driven agents," in *Proceedings of the AAAI Conference on*

Artificial Intelligence (AAAI-26), vol. 40, no. 48, 2026, pp. 41 694–41 696.

[23] Y. Han, J. Liu, A. Luo, Y. Wang, and S. Bao, "Fine-tuning LLM-Assisted Chinese disaster geospatial intelligence extraction and case studies," *ISPRS International Journal of Geo-Information*, vol. 14, no. 2, p. 79, 2025.

[24] T.-H. Yang, O. T. Wijaya, S. Ardianto, and A. B. Christian, "A novel multimodal large language model-based approach for urban flood detection using open-access closed circuit television in Bandung, Indonesia," *Water*, vol. 17, no. 18, p. 2739, 2025.

[25] H. Pang, M. K. Lo, A. Y. Leung, and S. Wu, "Reconstruction of landslide events in urban setting using LLM-based agentic AI with multimodal data," *Soils and Foundations*, vol. 66, no. 3, p. 101789, 2026.

[26] J. Zhu, P. Dang, Y. Cao, J. Lai, Y. Guo, P. Wang, and W. Li, "A flood knowledge-constrained large language model interactable with GIS: enhancing public risk perception of floods," *International Journal of Geographical Information Science*, vol. 38, no. 4, pp. 603–625, 2024.

[27] M. Ronco, L. Bandelli, L. Bertolini, S. Consoli, D. Delforge, A. Spadaro, M. Verile, and C. Corbane, "Disaster storylines and knowledge graphs from global news with large language models and retrieval-augmented generation," *Scientific Data*, vol. 13, no. 1, p. 689, 2026.

[28] E. Mandana, G. Vlahavas, and A. Vakali, "EVuLLM: Ethereum smart contract vulnerability detection using large language models," *Electronics*, vol. 14, no. 16, p. 3226, 2025.

[29] T. G. Dietterich, "Ensemble methods in machine learning," in *Multiple Classifier Systems (MCS 2000)*, ser. Lecture Notes in Computer Science, vol. 1857. Springer, 2000, pp. 1–15.

[30] L. Breiman, "Random forests," *Machine Learning*, vol. 45, no. 1, pp. 5–32, 2001.

[31] B. Lakshminarayanan, A. Pritzel, and C. Blundell, "Simple and scalable predictive uncertainty estimation using deep ensembles," in *Advances in Neural Information Processing Systems*, vol. 30, 2017.

[32] E. Hüllermeier and W. Waegeman, "Aleatoric and epistemic uncertainty in machine learning: an introduction to concepts and methods," *Machine Learning*, vol. 110, no. 3, pp. 457–506, 2021.

[33] R. El-Yaniv and Y. Wiener, "On the foundations of noise-free selective classification," *Journal of Machine Learning Research*, vol. 11, pp. 1605–1641, 2010.

[34] Y. Geifman and R. El-Yaniv, "Selective classification for deep neural networks," in *Advances in Neural Information Processing Systems*, vol. 30, 2017.

[35] F. Pedregosa *et al.*, "Scikit-learn: Machine learning in Python," *Journal of Machine Learning Research*, vol. 12, pp. 2825–2830, 2011.

[36] G. Ke, Q. Meng, T. Finley, T. Wang, W. Chen, W. Ma, Q. Ye, and T.-Y. Liu, "LightGBM: A highly efficient gradient boosting decision tree," in *Advances in Neural Information Processing Systems*, vol. 30, 2017.